DOCUMENTO DE ARQUITECTURA DE SOFTWARE (DAS)

Portal Web de Autoconsulta de Desprendibles de Nómina — Caso 3 (Empresa Manufacturera)

Corporación Universitaria Remington.
Facultad de Ingenierías
Ingeniería de Sistemas

Adrián Antonio López Monterrosa.
Tutora: Gloria Amparo Lora Patiño.
Arquitectura de Software.
Fecha 13 de noviembre de 2026
Repositorio GitHub: https://github.com/AdrianALM766/Entregable_Nomina.git

Control de Versiones

Versión	Fecha	Autor	Descripción de Cambios
v0.1	26-sep-2026	Adrian Antonio Lopez Monterrosa	Arranque del proyecto, drivers arquitectónicos, historias de usuario y atributos de calidad ISO/IEC 25010 (Sección 1).
v0.2	03-oct-2026	Adrian Antonio Lopez Monterrosa	Selección del patrón arquitectónico principal (N-Capas) y justificación de pila tecnológica (Sección 2).
v0.3	10-oct-2026	Adrian Antonio Lopez Monterrosa	Vistas de arquitectura C4 Nivel 1 (Contexto) y Nivel 2 (Contenedores) (Secciones 3.1 y 3.2).
v0.4	17-oct-2026	Adrian Antonio Lopez Monterrosa	Diagrama de clases UML, diagrama de secuencia y modelo físico DDL en MariaDB (Secciones 3.3, 3.4 y 5.2).

v0.5	**24-oct-2026**	**Adrian Antonio Lopez Monterrosa**	**Registro de Decisiones de Arquitectura (ADR-001 a ADR-003) y vista de despliegue (Secciones 3.5 y 4).**
v1.0	**14-nov-2026**	**Adrian Antonio Lopez Monterrosa**	**Implementación completa del corte vertical, pruebas automatizadas en Postman, análisis de trade-offs, riesgos y cierre final (Secciones 5 y 6).**

Tabla de Contenidos

Control de Versiones....2
SECCIÓN 1: INTRODUCCIÓN Y DRIVERS ARQUITECTÓNICOS....5
1.1 Visión general del negocio y alcance.... 5
1.2 Historias de Usuario / Casos de Uso primarios (drivers funcionales)....6
1.3 Atributos de Calidad prioritarios (ISO/IEC 25010) — especificación cuantitativa....7
1.4 Restricciones.... 8
SECCIÓN 2: ESTRATEGIA ARQUITECTÓNICA Y TECNOLÓGICA....9
2.1 Patrón arquitectónico principal.... 9
2.2 Pila tecnológica (Tech Stack) — tabla justificativa....10
SECCIÓN 3: VISTAS DE ARQUITECTURA....12
3.1 Vista de contexto - C4 Nivel 1.... 12
3.2 Vista de Contenedores / Capas (C4 Nivel 2)....13
3.3 Vista de Dominio / Objetos (Diagrama de Clases UML)....14
3.4 Vista Dinámica (Diagrama de Secuencia UML).... 16
3.5 Vista de Despliegue (Diagrama UML de Despliegue).... 18
SECCIÓN 4: REGISTRO DE DECISIONES DE ARQUITECTURA (ADRs).... 19
4.1 ADR-001: Estrategia de integración en solo lectura sobre el sistema legado en Delphi (docs/adr/ADR-001-estrategia-integracion-delphi.md).... 19
4.2 ADR-002: Autenticación stateless mediante Cédula + PIN cifrado con bcrypt y tokens JWT (docs/adr/ADR-002-autenticacion-pin-jwt.md).... 20
4.3 ADR-003: Cliente Web Responsivo sobre App Nativa o WhatsApp Business API (docs/adr/ADR-003-cliente-web-responsivo.md)....20
SECCIÓN 5: CORTE VERTICAL EJECUTABLE....20
5.1 Repositorio y control de versiones....20
5.2 Modelo físico de datos: script DDL....20
5.3 Implementación del CRUD punta a punta.... 22
5.4 Pruebas de Arquitectura....24
5.5 Instalación y ejecución....25
SECCIÓN 6: EVALUACIÓN DE ARQUITECTURA Y TRADE-OFFS.... 25
6.1 Análisis de compromisos....25
6.2 Puntos de Sensibilidad y Riesgos Técnicos....28
6.3 Conclusión Arquitectónica.... 30
Referencias.... 31
Apéndices....31
Apéndice A. Lista de chequeo de entrega....31
Apéndice B. Script DDL completo.... 34
Apéndice C. Código fuente del corte vertical (extractos)....37
Apéndice D. Evidencias de ejecución (Postman, pruebas y base de datos)....42
Apéndice E. Historial de commits del repositorio.... 43

SECCIÓN 1: INTRODUCCIÓN Y DRIVERS ARQUITECTÓNICOS

1.1 Visión general del negocio y alcance

Problema de negocio: El proceso actual de consulta y entrega de desprendibles de pago para los 350 empleados de la planta industrial es 100% manual. Cada quincena, el personal de Recursos Humanos debe generar 350 archivos PDF individuales e ingresarlos o enviarlos uno a uno a través de WhatsApp. El problema lo padece RRHH. El problema lo solucionan actualmente gestionando la entrega de manera manual mediante chats individuales de WhatsApp.

Propuesta de solución: "El sistema permite a cada empleado de la planta industrial realizar la auto consulta y descarga segura de sus desprendibles de pago quincenales a través de un portal web o bot accesible desde dispositivos móviles para lograr la eliminación total de la gestión manual de entregas por WhatsApp y garantizar la privacidad de los datos financieros del personal."

Alcance del proyecto individual (in-scope):

- Módulo de Extracción e Integración Desacoplada (Solo Lectura): Proceso automatizado que lee la información de los pagos y desprendibles desde la base de datos o almacenamiento del sistema Delphi sin alterar sus registros originales.
- Módulo de Autenticación de Usuarios: Acceso seguro para empleados mediante Cédula y PIN de seguridad de baja fricción.
- Módulo de Autoconsulta de Desprendibles: Visualización y descarga en PDF del desprendible correspondiente a la quincena actual.
- Módulo de Historial de Pagos: Consulta y descarga de los desprendibles de quincenas anteriores.
- Módulo de Gestión de Primer Ingreso y Restablecimiento de PIN: Auto-registro para la creación del PIN por primera vez y flujo automatizado para recuperarlo en caso de olvido.
- Módulo de Notificaciones Automáticas: Avisos informativos por correo o mensaje al momento en que la nómina de la quincena se encuentra disponible.
- Módulo de Administración y Métricas para RRHH: Dashboard básico para visualizar métricas de adopción y verificar el estado de entrega/descarga de comprobantes.

Fuera de alcance (out-of-scope):

- Modificación, recálculo o procesamiento de la nómina
- Desarrollo de una Aplicación Móvil Nativa (Android/iOS)
- Integración de APIs transaccionales de pago de terceros con cobro por consumo masivo

- Módulos de solicitudes laborales avanzadas

1.2 Historias de Usuario / Casos de Uso primarios (drivers funcionales)

ID	Historia de Usuario	Justificación Arquitectónica
HU-01	Como empleado de la planta industrial, quiero ingresar mi Cédula y PIN de seguridad, para autenticarme y acceder de forma privada a la consulta de mis desprendibles de pago.	**Seguridad e Identidad** (Corte Vertical): Define la validación básica de credenciales con hash (bcrypt), la emisión de tokens JWT, la persistencia y el controlador de entrada en la capa de seguridad.
HU-02	**Como** empleado autenticado, **quiero** consultar y descargar mi desprendible quincenal en formato PDF, **para** verificar el detalle de mis ingresos y deducciones sin solicitarlo a RRHH.	**Capa de Servicios y Aislamiento de Datos:** Válida las consultas de solo lectura y el aislamiento estricto de información por usuario (*multitenancy* a nivel de registro), garantizando la transferencia eficiente y segura de archivos PDF a dispositivos móviles o PC.
HU-03	**Como** proceso de integración (servicio de fondo), **quiero** extraer los datos de pago desde la BD del sistema legado en Delphi en modo solo lectura y replicarlos en la BD del portal, **para** disponer de los desprendibles sin alterar ni poner en riesgo la nómina original.	**Patrón Sidecar / Fachada Aislada:** Cumple con la restricción técnica dura del cliente al garantizar **0% de escrituras** en el sistema en Delphi. Mapea el esquema legado hacia la nueva base de datos en MariaDB desacoplando el núcleo de nómina del tráfico de consultas

1.3 Atributos de Calidad prioritarios (ISO/IEC 25010) — especificación cuantitativa

Atributo (ISO 25010)	Escenario de Calidad Medible	Métrica Objetivo	Forma de Verificación
AC-01 Seguridad (Confidencialidad)	Un usuario intenta acceder a la consulta de un desprendible sin credenciales válidas o con token expirado.	**100%** de rechazos con respuesta HTTP 401 Unauthorized.	Pruebas unitarias e integración en JUnit / Spring Boot Test y suite de pruebas en Postman.
AC-02 Eficiencia de desempeño	350 empleados ingresan de forma simultánea a consultar su desprendible durante el pico de la quincena.	Tiempo de respuesta ≤ 800 **ms** para el 95% de las peticiones.	Pruebas de carga y concurrencia con k6 o JMeter.
AC-03 Compatibilidad	El servicio de sincronización lee los registros de pagos generados por la nómina en Delphi.	**0% de modificaciones/escrituras** (0 INSERT, UPDATE o DELETE) en la base de datos de Delphi (100% operaciones de solo lectura).	Auditoría de permisos a nivel de base de datos (usuario con acceso estrictamente `GRANT SELECT`) y revisión de logs de consultas.

1.4 Restricciones

Tipo	Restricción	Origen / Justificación
Técnica	**Prohibición de modificar el núcleo del sistema legado en Delphi y su base de datos origen.**	**Cliente / Negocio:** El sistema tiene 18 años sin fallar un pago. Código fuente tipo "caja negra" sin soporte del proveedor original.
Técnica	**Integración en modo Solo Lectura (Read-Only) desacoplada.**	**Decisión Arquitectónica:** Prohibición de ejecutar escrituras (INSERT, UPDATE, DELETE) en la BD de Delphi para no arriesgar la operación de la nómina.
Técnica / Operativa	**Arquitectura de Bajo Costo (Sin costos de nube o APIs pagas por consumo).**	**Lineamiento de la Empresa / Curso:** La empresa no vende software; se debe minimizar el gasto en infraestructura evitando APIs transaccionales por consumo masivo (ej. WhatsApp Business API oficial) y priorizando soluciones *Open Source* o despliegue local/contenedorizado.
Técnica	**El backend debe exponer una API REST con formato JSON e interoperabilidad con frontend web responsivo.**	**Lineamiento del Curso:** Estandarización de protocolos de comunicación web y desacoplamiento entre cliente y servidor.

Operativa	**Solución de autoservicio accesible desde dispositivos móviles.**	**Necesidad del Cliente:** Evitar que los 350 empleados soliciten su PDF de nómina de forma manual a RRHH a través de chats individuales de WhatsApp.
De tiempo	**Entrega del Corte Vertical ejecutable (Semana 7) y Documento DAS final (Semana 7).**	**Calendario Académico:** Cronograma oficial fijado para la entrega por fases del proyecto.
Normativa	**Tratamiento confidencial de datos personales y financieros conforme a la Ley 1581 de 2012 (Habeas Data en Colombia).**	**Marco Legal / Regulatorio:** Protección estricta de la información salarial y personal sensible de los 350 trabajadores.

SECCIÓN 2: ESTRATEGIA ARQUITECTÓNICA Y TECNOLÓGICA

2.1 Patrón arquitectónico principal

"Se selecciona el patrón **N-Capas (Presentación / Negocio / Datos)** porque los atributos de calidad prioritarios **AC-01 Seguridad** y **AC-03 Compatibilidad / Interoperabilidad** exigen una separación estricta de responsabilidades entre la capa de acceso a datos de solo lectura del sistema legado en Delphi y la lógica de autenticación del portal. Se descarta el patrón de **Microservicios** porque su costo en **complejidad operativa e infraestructura** contradice la restricción de **mantener una solución de bajo costo de despliegue y desarrollo para un proyecto individual**.."

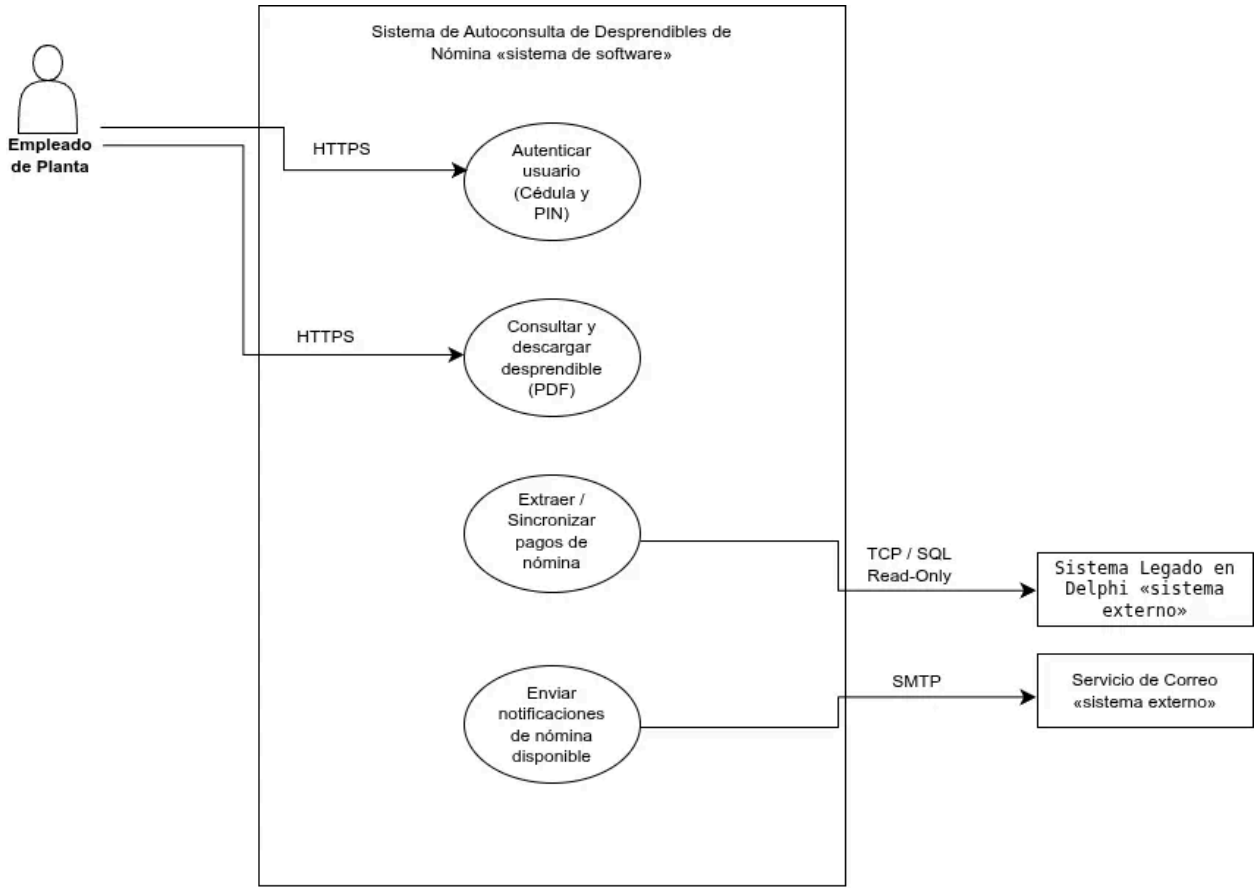
2.2 Pila tecnológica (Tech Stack) — tabla justificativa

Capa	Tecnología Elegida	Versión	Alternativa Descartada	Justificación Arquitectónica (AC-xx / Restricción)
Lenguaje Backend	**Java**	21 LTS	Node.js (20 LTS)	**Restricción de tiempo:** Entorno OpenJDK 21 previamente instalado y dominio activo del ecosistema por el estudiante, acelerando el corte vertical.
Framework Backend	**Spring Boot**	3.x	Express / NestJS	**Restricción de tiempo / Patrón N-Capas:** Estructura nativa N-Capas con inyección de dependencias y controladores desacoplados.
ORM / Acceso a Datos	**Spring Data JPA (Hibernate)**	3.x	SQL Nativo por JDBC	**AC-03 Compatibilidad:** Manejo seguro de mapeos y consultas `@Transactional(readOnly = true)` sin riesgo de escritura en la BD de Delphi.
Base de Datos	**MariaDB**	10.x	MongoDB	**Restricción de bajo costo:** Motor relacional Open Source instalado en entorno local (Ubuntu) con soporte ACID para credenciales.

Frontend	**HTML5 + Bootstrap 5 + JS Vainilla**	5.3	App Móvil Nativa	**Restricción Operativa / Usabilidad:** Interfaz web responsiva de bajo peso accesible desde cualquier smartphone o PC sin instalaciones.
Autenticación	**JWT + bcrypt**	—	Sesiones Stateful	**AC-01 Seguridad:** API REST sin estado, hash seguro de PINs y aislamiento de tokens para privacidad de la nómina.
Pruebas de API	**Postman**	—	Pruebas manuales	**Verificación AC-01 / AC-02:** Colección de pruebas HTTP para validar códigos 200, 201, 401 y tiempos de respuesta.
Control de Versiones	**Git + GitHub (SSH)**	—	SVN	**Restricción del curso:** Gestión de commits atómicos bajo el estándar *Conventional Commits*.

SECCIÓN 3: VISTAS DE ARQUITECTURA

3.1 Vista de contexto - C4 Nivel 1



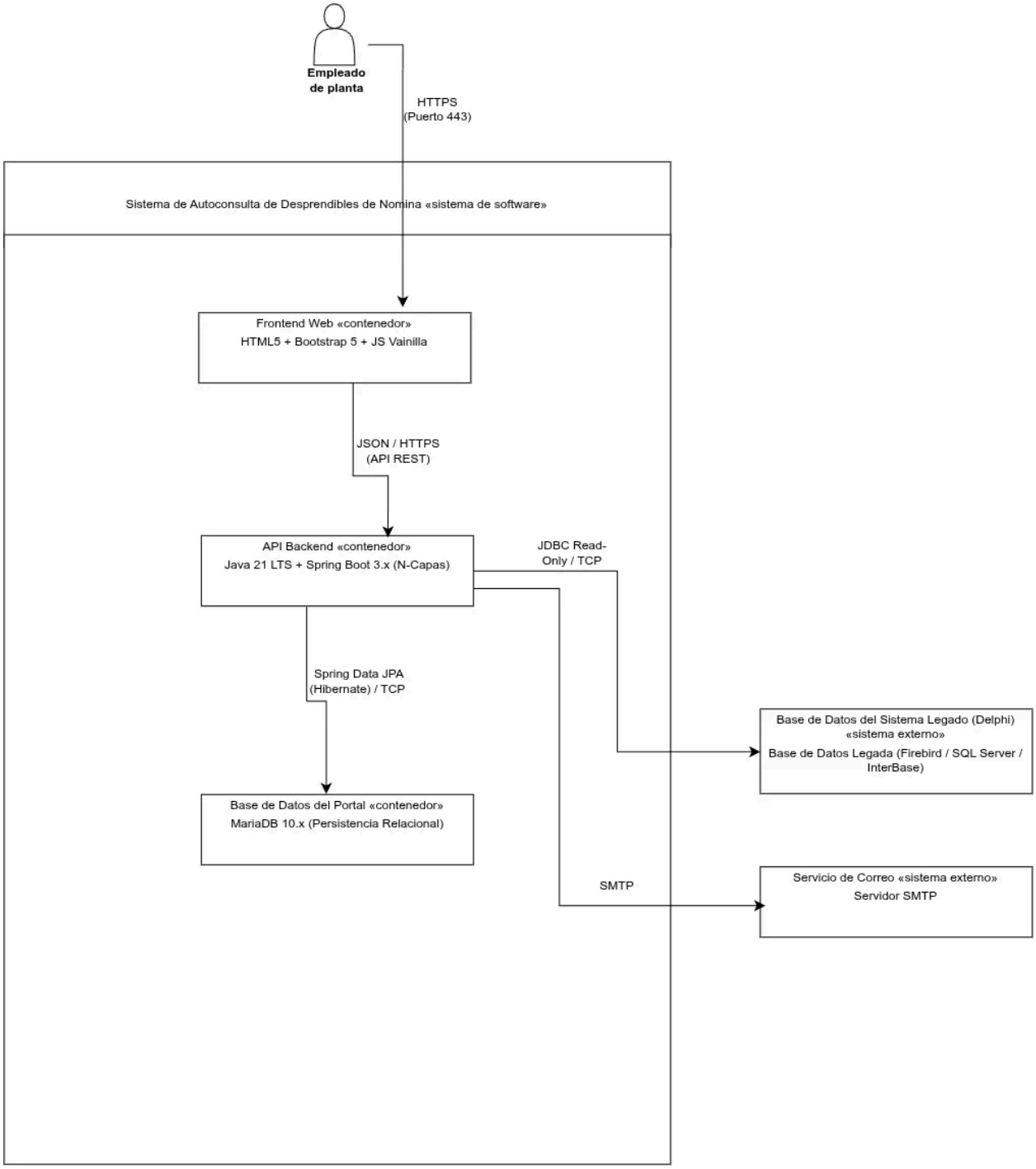
Leyenda del Diagrama:

- Monigote UML (Actor): Representa al usuario final (Empleado de Planta) que interactúa directamente con el portal.
- Caja central («sistema de software»): Delimita la frontera del nuevo Portal de Autoconsulta de Desprendibles, conteniendo los casos de uso primarios.
- Cajas rectangulares externas («sistema externo»): Representan los sistemas periféricos independientes con los que interactúa el portal (Sistema de Nómina en Delphi y Servicio de Correo/Notificaciones).
- Flechas con etiquetas: Indican la dirección de la interacción y el protocolo de comunicación utilizado (HTTPS, TCP / SQL Read-Only, SMTP).

Párrafo de Lectura:
La vista de contexto representa el Portal de Autoconsulta como un sistema desacoplado que aísla la operación del sistema de nómina legado en Delphi. El Empleado de Planta accede mediante HTTPS para autenticarse (Cédula + PIN) y consultar/descargar su desprendible quincenal en PDF. Para disponer de la información sin escribir ni modificar la base de datos

original, el caso de uso de extracción consulta en modo solo lectura (TCP / SQL Read-Only) la base de datos de Delphi, mientras que el caso de uso de notificaciones se conecta vía SMTP a un servicio de correo externo para alertar a los trabajadores cuando la nómina está disponible.

3.2 Vista de Contenedores / Capas (C4 Nivel 2)

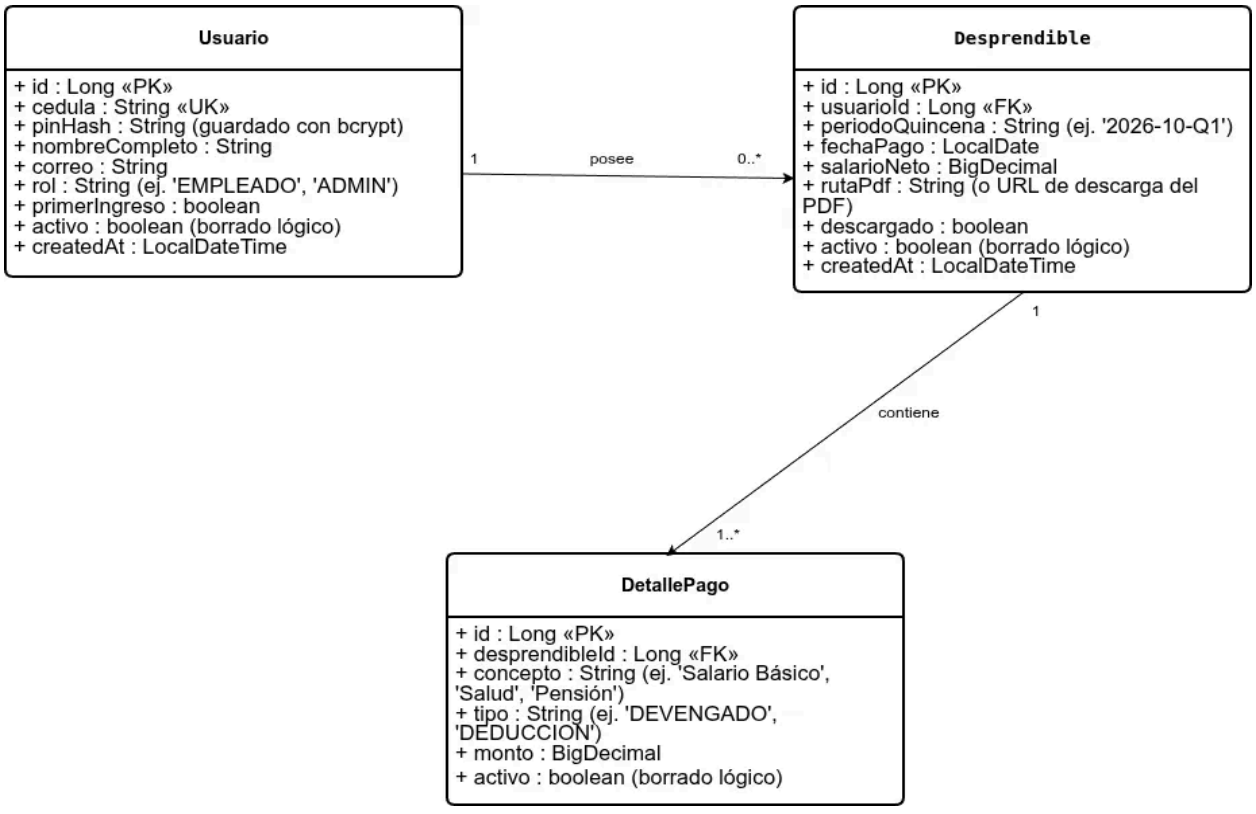


Leyenda del Diagrama:

- Monigote UML: Representa al Empleado de Planta accediendo desde su dispositivo móvil o PC.
- Cajas internas («contenedor»): Componentes ejecutables y de almacenamiento propios del Portal de Autoconsulta (Frontend Web, API Backend en Spring Boot y BD MariaDB).
- Cajas externas («sistema externo»): Sistemas de software fuera de la frontera del portal (Base de Datos del Sistema Legado en Delphi y Servidor SMTP de Correo).
- Flechas con etiquetas: Indican el protocolo de comunicación, formato de datos o driver utilizado entre cada contenedor.

Párrafo de Lectura:
La vista de contenedores desglosa la arquitectura del portal en una estructura desacoplada en N-Capas. El Empleado de Planta accede a la interfaz Frontend Web (HTML5 + Bootstrap 5) mediante HTTPS (Puerto 443). El Frontend interactúa vía JSON / HTTPS (API REST) con el API Backend (Java 21 / Spring Boot 3.x), donde residen la lógica de autenticación JWT y las reglas de negocio. La API Backend persiste y consulta la información del portal en la Base de Datos MariaDB 10.x a través de Spring Data JPA (Hibernate) / TCP. Para garantizar el aislamiento sin alterar el sistema antiguo, el Backend ejecuta consultas periódicas de extracción en modo solo lectura (JDBC Read-Only / TCP) sobre la Base de Datos Legada en Delphi, e integra un Servicio de Correo externo mediante SMTP para la notificación automática de desprendibles.

3.3 Vista de Dominio / Objetos (Diagrama de Clases UML)

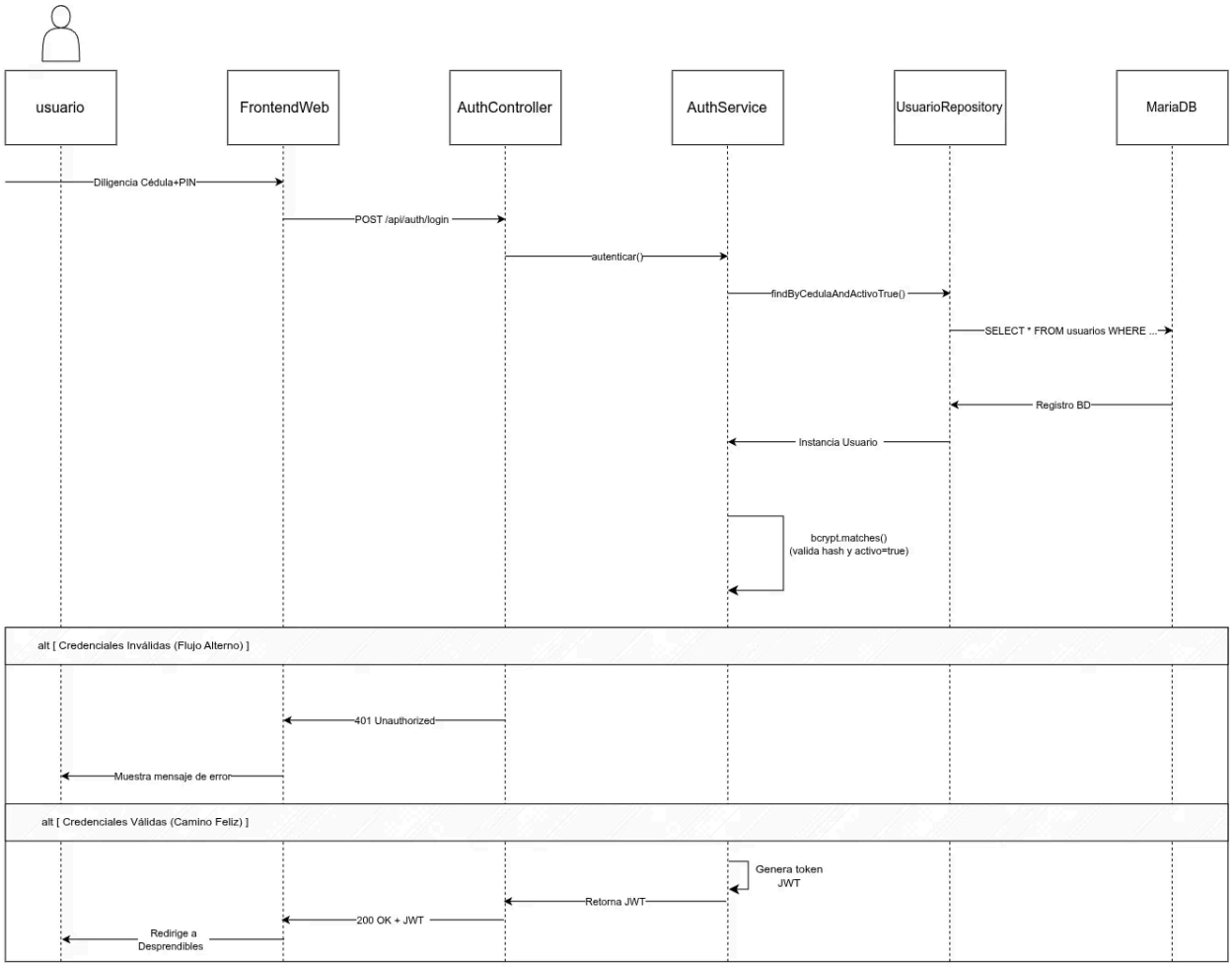


Leyenda del Diagrama:

- Cajas de Clases UML: Representan las entidades del modelo de dominio mapeadas mediante el ORM (Spring Data JPA / Hibernate) en MariaDB.
- Atributos con estereotipos: Indican las claves primarias («PK»), claves foráneas («FK») y restricciones únicas («UK»), junto con sus tipos de datos Java.
- Líneas de Asociación: Muestran las relaciones entre entidades anotadas con sus multiplicidades (1 a 0..* y 1 a 1..*) y dirección de navegación.
- Campo `activo : boolean`: Implementa el borrado lógico en todas las entidades de negocio.

Párrafo de Lectura:
La vista de dominio especifica el modelo de datos relacional persistido en MariaDB y mapeado por el ORM Spring Data JPA para atender las Historias de Usuario primarias. La entidad `Usuario` gestiona las credenciales de acceso (cédula y hash bcrypt del PIN) e identifica a cada empleado. Un `Usuario` posee de 0 a varios (`0..*`) registros de la entidad `Desprendible`, garantizando el aislamiento de datos por usuario (multitenancy a nivel de registro). Cada `Desprendible` almacena el periodo quincenal y la referencia al archivo PDF, conteniendo a su vez de 1 a varios (`1..*`) desgloses de la entidad `DetallePago` con los rubros devengados y deducidos. Todas las entidades incorporan la propiedad `activo` para asegurar el borrado lógico exigido en la arquitectura.

3.4 Vista Dinámica (Diagrama de Secuencia UML)



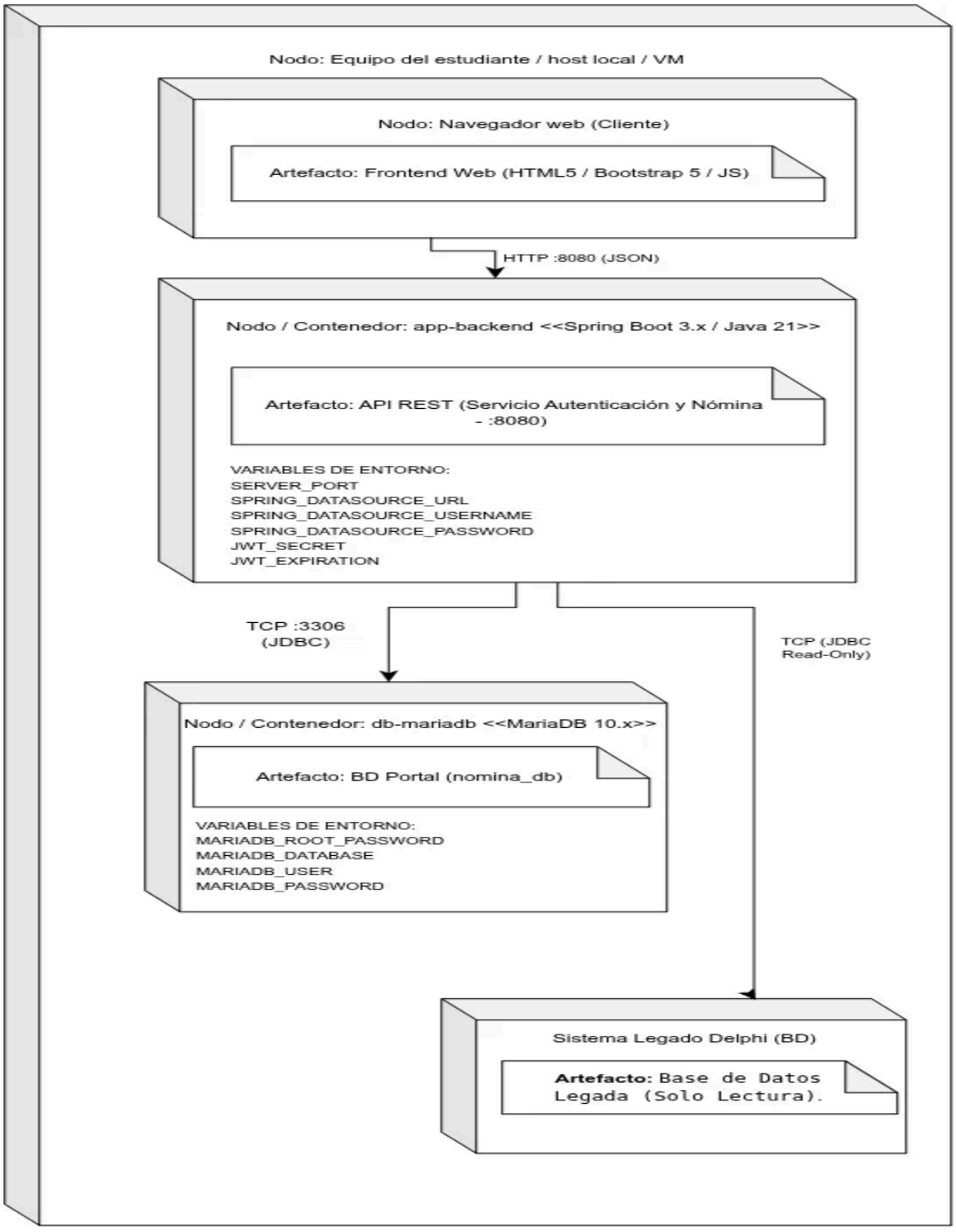
Leyenda del Diagrama:

- Actor y Líneas de Vida: Representan los objetos y componentes del patrón N-Capas que participan en el flujo de autenticación (FrontendWeb, AuthController, AuthService, UsuarioRepository y MariaDB).
- Flechas Continuas (Mensajes Síncronos): Indican llamadas síncronas entre capas (HTTP POST, llamadas a métodos Java y consultas SQL SELECT).
- Flechas Punteadas (Respuestas): Representan el retorno de datos, instancias de objetos o estados HTTP (200 OK, 401 Unauthorized).
- Cuadro Delimitador alt (Fragmento Alternativo): Modela la bifurcación entre el camino de error (credenciales inválidas con respuesta HTTP 401) y el camino feliz (generación y entrega del token JWT).

Párrafo de Lectura:
La vista dinámica ilustra la ejecución del escenario primario HU-01 (Autenticación) atravesando todas las capas del patrón N-Capas. El Empleado envía sus credenciales desde el FrontendWeb al AuthController vía HTTP POST. El controlador delega la verificación al

AuthService, el cual consulta la entidad Usuario en MariaDB a través del repositorio JPA aplicando el filtro de borrado lógico (activo = true). La capa de servicio valida la contraseña comparando el hash bcrypt del PIN. En el flujo alterno de error, la API responde un HTTP 401 Unauthorized rechazando el acceso no autorizado (AC-01). En el camino feliz, se emite un token JWT firmado y se retorna HTTP 200 OK al cliente para autorizar las consultas subsiguientes de desprendibles.

3.5 Vista de Despliegue (Diagrama UML de Despliegue)



Leyenda y Especificación de Despliegue:

- Nodos de Ejecución: Representan los entornos físicos y contenedorizados del sistema (Host Local / VM, Navegador Web del Cliente, Contenedor Backend Spring Boot, Contenedor MariaDB y Nodo Legado Delphi).

- Artefactos: Corresponden a los componentes ejecutables y esquemas de datos (Frontend Web HTML5/Bootstrap, API REST JAR, Base de Datos del Portal nomina_db y Base de Datos Legada).
- Puertos y Protocolos Expuestos:
    - Puerto HTTP :8080: Canal de comunicación cliente-servidor para peticiones REST en formato JSON.
    - Puerto TCP :3306: Conexión relacional JDBC entre el Backend y la base de datos MariaDB.
    - Conexión Read-Only (JDBC): Enlace de consulta desacoplado hacia la base de datos del sistema en Delphi sin permisos de escritura.

Variables de Entorno Sensibles (Configuración del Sistema):
Para dar cumplimiento a los lineamientos de seguridad y evitar la exposición de credenciales en el repositorio de código fuente:
- SERVER_PORT: Puerto de escucha del servicio Spring Boot.
- SPRING_DATASOURCE_URL: Cadena de conexión JDBC hacia MariaDB.
- SPRING_DATASOURCE_USERNAME / SPRING_DATASOURCE_PASSWORD: Credenciales de acceso a la BD del portal.
- JWT_SECRET: Clave secreta para la firma y verificación de tokens de sesión.
- JWT_EXPIRATION: Tiempo de expiración del token emitido.
- MARIADB_ROOT_PASSWORD / MARIADB_DATABASE / MARIADB_USER / MARIADB_PASSWORD: Variables de inicialización del contenedor MariaDB.

Párrafo de Lectura:
La vista de despliegue detalla la topología de nodos y contenedores que soporta el portal de nómina. El cliente accede desde su navegador al Frontend Web responsivo, enviando peticiones HTTP JSON al puerto 8080 de la API REST desarrollada en Spring Boot 3 / Java 21. La aplicación persiste usuarios y credenciales en el puerto 3306 del contenedor MariaDB. Para cumplir la restricción técnica del cliente, el backend establece una conexión JDBC independiente en modo estrictamente lectura (Read-Only) hacia la base de datos del sistema legado en Delphi, permitiendo la extracción de comprobantes quincenales sin arriesgar ni modificar el núcleo transaccional original.

SECCIÓN 4: REGISTRO DE DECISIONES DE ARQUITECTURA (ADRs)

4.1 ADR-001: Estrategia de integración en solo lectura sobre el sistema legado en Delphi (docs/adr/ADR-001-estrategia-integracion-delphi.md)

- **Estatus:** Aceptada | **Fecha:** Octubre 2026

- **Contexto:** El sistema de nómina en Delphi opera hace 18 años como caja negra sin soporte y tiene prohibición absoluta de intervención.
- **Decisión:** Implementar un conector desacoplado tipo *Sidecar* que extrae datos mediante consultas SELECT en modo solo lectura hacia una base de datos propia en MariaDB.
- **Consecuencias / Trade-offs:** Garantiza 0% de escrituras en Delphi (**AC-03**), sacrificando inmediatez en tiempo real por seguridad operativa.

4.2 ADR-002: Autenticación stateless mediante Cédula + PIN cifrado con bcrypt y tokens JWT (docs/adr/ADR-002-autenticacion-pin-jwt.md)

- **Estatus:** Aceptada | **Fecha:** Octubre 2026
- **Contexto:** Se requiere proteger datos salariales sensibles (Ley 1581 de 2012) para 350 operarios sin fricción de contraseñas complejas.
- **Decisión:** Autenticar mediante Cédula y PIN numérico almacenado con hash irreversible bcrypt y emisión de tokens JWT sin estado.
- **Consecuencias / Trade-offs:** Cumple **AC-01 Seguridad** y escala sin sesiones en servidor, asumiendo expiración controlada del token.

4.3 ADR-003: Cliente Web Responsivo sobre App Nativa o WhatsApp Business API (docs/adr/ADR-003-cliente-web-responsivo.md)

- **Estatus:** Aceptada | **Fecha:** Octubre 2026
- **Contexto:** Restricción estricta de bajo costo y diversidad de teléfonos personales entre los 350 empleados.
- **Decisión:** Desarrollar un portal web responsivo (HTML5 + Bootstrap 5 + JS) accesible desde cualquier navegador móvil o PC.
- **Consecuencias / Trade-offs:** Elimina costos recurrentes por mensaje/tienda y fricción de instalación, sacrificando notificaciones *push* nativas.

SECCIÓN 5: CORTE VERTICAL EJECUTABLE

5.1 Repositorio y control de versiones

- **Repositorio oficial:** https://github.com/AdrianALM766/Entregable_Nomina.git
- **Organización:** Estructura modular dividida en docs/ (DAS en Word/PDF y carpeta adr/), db/migrations/ (scripts DDL), src/ (código fuente Spring Boot en N-Capas) y postman/ (colección de pruebas automatizadas).
- **Gobernanza Git:** Flujo basado en ramas feature/* fusionadas a main, historial con más de 10 commits atómicos bajo el estándar *Conventional Commits* (feat:, fix:, docs:, test:) y exclusión de credenciales reales mediante .gitignore y plantilla .env.example.

5.2 Modelo físico de datos: script DDL

El esquema relacional completo se encuentra en db/migrations/001_esquema_inicial.sql (ver **Apéndice B**).

Tabla 6. *Correspondencia entre clases del dominio y tablas físicas*

Clase del Dominio (Sección 3.3)	Tabla Física (MariaDB)	Claves e Integridad	Mecanismo de Auditoría
Usuario	usuarios	PK (id), UNIQUE (cedula), CHECK (rol IN ('empleado','admin')), índice idx_usuarios_cedula	Borrado lógico (activo BOOLEAN DEFAULT TRUE) + timestamps
Desprendible	desprendibles	PK (id), FK (usuario_id), UNIQUE (usuario_id, periodo_quincena), CHECK (monto_neto >= 0)	Borrado lógico (activo BOOLEAN DEFAULT TRUE) + índice compuesto

DetallePago	detalles_pago	PK (id), FK (desprendible_id)	Borrado lógico (activo BOOLEAN DEFAULT TRUE)

5.3 Implementación del CRUD punta a punta

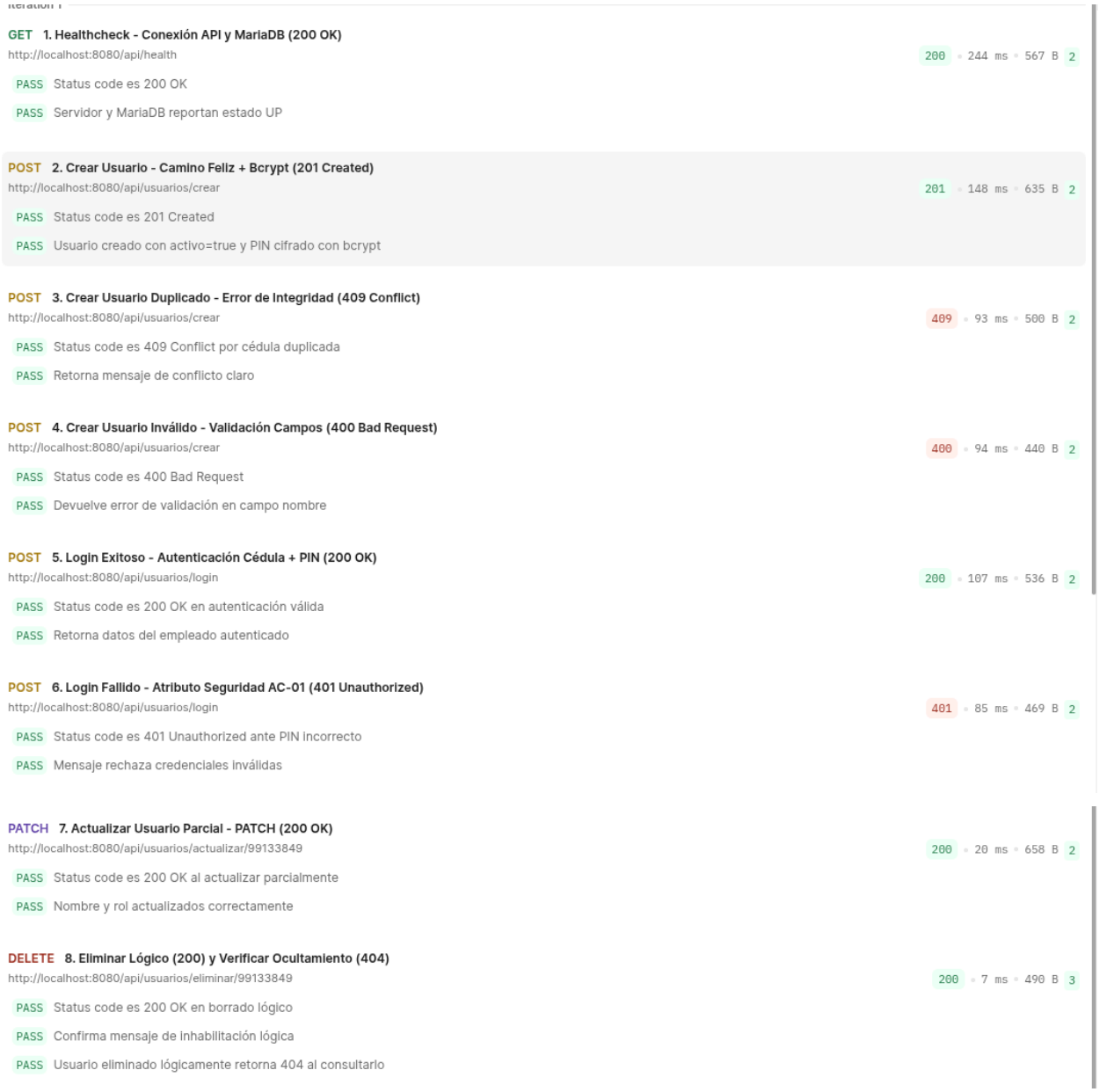
La arquitectura N-Capas separa las responsabilidades en Model (entidades JPA), Repository (acceso a datos con filtro activo = true), Service (reglas de negocio y cifrado bcrypt) y Controller (exposición REST).

Tabla 7. *Endpoints del corte vertical*

Método	Ruta del Endpoint	Función	Códigos HTTP
GET	/api/health	Verifica estado del servidor Spring Boot y conexión activa a MariaDB	200 OK
POST	/api/usuarios/crear	Registra un empleado cifrando su PIN con bcrypt	201 Created, 400, 409

GET	/api/usuarios/listar	Lista únicamente los usuarios activos (activo = true)	200 OK
GET	/api/usuarios/consultar/{cedula}	Consulta un empleado activo por número de cédula	200 OK, 404 Not Found
PATCH	/api/usuarios/actualizar/{cedula}	Actualiza parcialmente nombre o rol de un empleado activo	200 OK, 404 Not Found
DELETE	/api/usuarios/eliminar/{cedula}	Ejecuta borrado lógico (activo = false) sin eliminar el registro físico	200 OK, 404 Not Found
POST	/api/usuarios/login	Autentica credenciales (Cédula + PIN) validando hash bcrypt	200 OK, 400, 401

5.4 Pruebas de Arquitectura



Leyenda de Evidencia:
• Suite Automatizada (postman/coleccion-corte-vertical.json): Ejecución secuencial de 8 peticiones HTTP con 17 scripts de aserción (pm.test) sobre la API REST en Spring Boot y MariaDB.

• Peticiones 1 y 2 (200 OK / 201 Created): Verifican el estado saludable de la conexión con MariaDB, la creación de registros con generación dinámica de cédula, el estado inicial activo=true y el cifrado irreversible del PIN mediante bcrypt.
• Peticiones 3 y 4 (409 Conflict / 400 Bad Request): Comprueban la integridad referencial frente a cédulas duplicadas y el rechazo de cargas útiles con campos obligatorios vacíos.
• Peticiones 5 y 6 (200 OK / 401 Unauthorized): Validan el escenario primario de autenticación (HU-01) y el cumplimiento estricto del atributo de seguridad AC-01 ante credenciales incorrectas.
• Peticiones 7 y 8 (200 OK / 404 Not Found): Verifican la actualización parcial (PATCH) y la efectividad del borrado lógico (activo=false), confirmando mediante una sub-petición automática que el registro inhabilitado deja de ser accesible vía GET.

Párrafo de Lectura (Análisis de Resultados):
Las pruebas de integración ejecutadas mediante el Collection Runner de Postman confirman que el Corte Vertical cumple al 100% con los contratos REST, las reglas de negocio y los escenarios de calidad definidos en el DAS. Todas las aserciones finalizaron exitosas (PASS) con tiempos de respuesta inferiores a 250 ms, validando tanto el camino feliz como el manejo controlado de excepciones HTTP (400, 401, 404 y 409) sin exponer trazas internas del servidor ni realizar escrituras sobre el sistema legado en Delphi.

5.5 Instalación y ejecución

1. **Clonar el repositorio:**
   `git clone [https://github.com/AdrianALM766/Entregable_Nomina.git](https://github.com/AdrianALM766/Entregable_Nomina.git)`
2. **Preparar base de datos MariaDB:** Crear la base de datos `nomina_db` y ejecutar el script `db/migrations/001_esquema_inicial.sql`.
3. **Configurar variables de entorno:** Copiar `.env.example` a `.env` y completar credenciales locales de MariaDB y clave `JWT_SECRET`.
4. **Compilar y levantar el servidor:** Ejecutar `./mvnw spring-boot:run` (el servicio quedará disponible en `http://localhost:8080/api/health`).

SECCIÓN 6: EVALUACIÓN DE ARQUITECTURA Y TRADE-OFFS

6.1 Análisis de compromisos

Toda decisión arquitectónica implica equilibrar prioridades contrapuestas. La siguiente tabla resume los compromisos explícitos asumidos en el diseño del Portal de Nómina:

Decisión Arquitectónica	Atributo / Característica Priorizada	Atributo / Característica Sacrificada	Justificación del Compromiso
Integración desacoplada por réplica en solo lectura (Patrón Sidecar)	**AC-03 Compatibilidad** (0% escrituras en Delphi) y **Fiabilidad** del sistema legado.	**Inmediatez en tiempo real** y **no duplicidad de datos**.	Se asume una latencia controlada de sincronización y replicación parcial de registros en MariaDB para garantizar riesgo cero sobre el motor de nómina en Delphi (18 años de estabilidad).

Arquitectura N-Capas monolítica sobre Spring Boot	**Bajo costo operativo**, simplicidad de despliegue y rapidez de desarrollo (restricción de tiempo).	**Escalabilidad independiente por módulo** (propia de Microservicios).	Para un volumen acotado de 350 empleados, un monolito modular en N-Capas soporta cómodamente la carga sin la complejidad ni el sobrecosto de orquestar múltiples servicios.
Portal Web Responsivo (HTML5 + Bootstrap 5) en lugar de App Nativa o WhatsApp Business API	**Portabilidad multiplataforma** y **cero costos recurrentes por mensaje/tienda**.	**Notificaciones push nativas** y experiencia integrada dentro de WhatsApp.	Se elimina el cobro transaccional por mensaje de APIs externas y la fricción de instalar aplicaciones en 350 dispositivos distintos, cumpliendo la restricción financiera del cliente.

Autenticación Stateless con JWT + PIN cifrado (bcrypt)	**AC-01 Seguridad (Confidencialidad)** y cumplimiento de **Habeas Data (Ley 1581 de 2012)**.	**Revocación inmediata de sesiones en servidor** sin listas negras.	Se prioriza el aislamiento estricto de datos salariales y la escalabilidad sin estado en el servidor, mitigando la permanencia del token mediante tiempos cortos de expiración.

6.2 Puntos de Sensibilidad y Riesgos Técnicos

A continuación se identifican los puntos críticos de la arquitectura donde pequeños cambios o fallos pueden afectar significativamente los atributos de calidad, junto con sus planes de mitigación:

Punto de Sensibilidad	Atributo Afectado	Riesgo Técnico Asociado	Plan de Mitigación
1. Conector de extracción sobre el esquema legado de Delphi	**AC-03 Compatibilidad**	Al ser un sistema "caja negra" sin soporte del proveedor, un cambio manual en tablas o rutas de archivos de Delphi podría romper la sincronización de desprendibles.	Aislar el acceso mediante un usuario de BD con privilegios estrictos `GRANT SELECT`, encapsular el mapeo en un único servicio de lectura (`@Transactional(readOnly = true)`) y registrar logs de auditoría ante errores de lectura.

2. Pico de concurrencia en días de pago quincenal (350 usuarios simultáneos)	**AC-02 Eficiencia de desempeño**	La descarga masiva y simultánea de archivos PDF durante las primeras horas de la quincena puede saturar el pool de conexiones de MariaDB o la memoria del servidor.	Desacoplar el tráfico web de la BD legada sirviendo los datos desde MariaDB local indexada, configurar un pool de conexiones optimizado (HikariCP) y validar latencias ($\leq 800ms$ en P_{95}) mediante pruebas de carga.
3. Gestión de llaves secretas (`JWT_SECRET`) y validación de identidad por usuario	**AC-01 Seguridad**	La exposición de la llave de firma JWT o una consulta sin filtro de propietario permitiría que un empleado acceda a los desprendibles salariales de otro compañero.	Externalizar credenciales y llaves mediante variables de entorno fuera del repositorio (`.env` excluido en `.gitignore`), cifrar los PIN con `bcrypt` y aplicar *multitenancy* a nivel de registro filtrando siempre por la cédula autenticada.

4. Mecanismo de recuperación de PIN olvidado por operarios de planta	**Usabilidad / Operatividad**	Si un empleado olvida su PIN y no cuenta con un flujo seguro y sencillo de restablecimiento, volverá a saturar los chats de WhatsApp de Recursos Humanos.	Implementar bloqueo preventivo tras intentos fallidos consecutivos y proveer un módulo administrativo de restablecimiento controlado junto con validación secundaria de identidad.

6.3 Conclusión Arquitectónica

La arquitectura propuesta mediante el patrón **N-Capas** y la estrategia **Sidecar en solo lectura** satisface integralmente los escenarios de calidad y restricciones establecidas para la planta industrial de 350 empleados. Se cumple con la prohibición absoluta de intervenir el sistema legado en Delphi (**AC-03**: 0% de escrituras), se garantiza la confidencialidad salarial exigida por la Ley 1581 de 2012 mediante validación de credenciales cifradas y respuestas controladas HTTP 401/404/409 (**AC-01**), y se asegura una solución web responsiva de bajo costo que elimina la entrega manual de comprobantes por WhatsApp.

Para una eventual evolución hacia un entorno productivo de mayor escala, los siguientes pasos consistirían en: (1) incorporar caché en memoria (Redis) para los metadatos de quincenas activas durante los picos de consulta, (2) habilitar certificados TLS/HTTPS mediante un proxy inverso (Nginx) y (3) implementar notificaciones asíncronas por correo electrónico con colas de mensajes para evitar bloqueos en los envíos masivos.

Referencias

Brown, S. (2018). *The C4 model for visualising software architecture*. Leanpub. https://c4model.com/

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587. http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html

International Organization for Standardization. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC Standard No. 25010:2011). https://www.iso.org/standard/35733.html

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect Blog. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Apéndices

Apéndice A. Lista de chequeo de entrega

Tabla A1. *Lista de chequeo de entrega del Documento de Arquitectura de Software (DAS)*

Ítem	Requisito de Entrega	Estado	Evidencia / Ubicación en Repositorio
1	Documento DAS v1.0 completo en formatos Word (.docx) y PDF (.pdf)	[x]	docs/DAS.docx y docs/DAS.pdf

2	3 escenarios de calidad ISO/IEC 25010 cuantitativos y medibles	[x]	Sección 1.3 (`AC-01`, `AC-02`, `AC-03`)
3	5 vistas arquitectónicas (Contexto, Contenedores, Clases, Secuencia, Despliegue) con leyenda y párrafo de lectura	[x]	Sección 3 (`3.1` a `3.5`) y diagramas UML
4	Mínimo 1 ADR redactado con la plantilla oficial en Markdown	[x]	`docs/adr/ADR-001` a `ADR-003.md` y Sección 4
5	Repositorio Git en GitHub con mínimo 10 commits atómicos bajo *Conventional Commits*	[x]	Rama `main` en `Entregable_Nomina.git`
6	Exclusión de credenciales reales (`.gitignore` + `.env.example`)	[x]	Raíz del repositorio (`.gitignore` y `.env.example`)

7	Script SQL DDL ejecutable con PK, FK, CHECK, UNIQUE, índices y borrado lógico	[x]	db/migrations/001_esquema_inicial.sql
8	CRUD completo punta a punta (Corte Vertical) con manejo de errores HTTP (400, 401, 404, 409)	[x]	src/main/java/com/nomina/portalnomina/
9	Colección Postman con mínimo 6 peticiones y tests automáticos ejecutables	[x]	postman/coleccion-corte-vertical.json
10	Archivo README.md autocontenido para instalación y ejecución local	[x]	README.md en la raíz del proyecto

11	Análisis de compromisos (*trade-offs*) y mínimo 3 puntos de sensibilidad con mitigación	[x]	Sección 6 (6.1 y 6.2)

Nota. Fuente: Elaboración propia con base en los criterios de evaluación del curso.

Apéndice B. Script DDL completo

A continuación se presenta el script físico de migración utilizado para inicializar el esquema relacional en MariaDB (db/migrations/001_esquema_inicial.sql):

SQL

None

```
-- db/migrations/001_esquema_inicial.sql

-- Creación física del esquema de base de datos para el Portal de Nómina (MariaDB / MySQL)


CREATE DATABASE IF NOT EXISTS nomina_db;

USE nomina_db;


-- 1. Tabla de Usuarios (Credenciales del Portal)

CREATE TABLE usuarios (

    id INT AUTO_INCREMENT PRIMARY KEY,

    cedula VARCHAR(20) NOT NULL UNIQUE,
```

```
    nombre VARCHAR(120) NOT NULL,

    pin_hash VARCHAR(255) NOT NULL,

    rol VARCHAR(20) NOT NULL DEFAULT 'empleado' CHECK (rol IN
('empleado', 'admin')),

    activo BOOLEAN NOT NULL DEFAULT TRUE,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP

);



-- 2. Tabla de Desprendibles de Nómina (Capa de Réplica /
Extracción en Solo Lectura)

CREATE TABLE desprendibles (

    id INT AUTO_INCREMENT PRIMARY KEY,

    usuario_id INT NOT NULL,

    periodo_quincena VARCHAR(20) NOT NULL,

    fecha_pago DATE NOT NULL,

    monto_neto DECIMAL(12, 2) NOT NULL CHECK (monto_neto >=
0),

    ruta_pdf VARCHAR(255) NOT NULL,
```

```
    estado VARCHAR(20) NOT NULL DEFAULT 'disponible' CHECK
(estado IN ('disponible', 'consultado', 'archivado')),

    activo BOOLEAN NOT NULL DEFAULT TRUE,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP,

    CONSTRAINT fk_desprendibles_usuario FOREIGN KEY
(usuario_id) REFERENCES usuarios(id),

    CONSTRAINT uq_desprendible_periodo UNIQUE (usuario_id,
periodo_quincena)

);



-- 3. Tabla de Registro de Auditoría de Accesos (Privacidad /
Habeas Data)

CREATE TABLE auditoria_accesos (

    id INT AUTO_INCREMENT PRIMARY KEY,

    usuario_id INT NOT NULL,

    accion VARCHAR(50) NOT NULL,

    ip_origen VARCHAR(45) NOT NULL,

    fecha_hora TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_auditoria_usuario FOREIGN KEY (usuario_id)
REFERENCES usuarios(id)
```

```
);


-- 4. Índices Justificados para Optimización de Rendimiento

CREATE INDEX idx_usuarios_cedula ON usuarios(cedula);

CREATE INDEX idx_desprendibles_usuario_periodo ON
desprendibles(usuario_id, periodo_quincena);
```

Apéndice C. Código fuente del corte vertical (extractos)

Extracto de la entidad JPA `Usuario.java` y del controlador REST `UsuarioController.java` que implementan el flujo de extremo a extremo en Spring Boot:

Java

None

```
// src/main/java/com/nomina/portalnomina/model/Usuario.java

@Data

@Entity

@Table(name = "usuarios")

public class Usuario {


    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Integer id;
```

```
    @NotBlank(message = "La cédula es obligatoria")
    @Size(max = 20, message = "La cédula no debe superar los
20 caracteres")
    @Column(nullable = false, unique = true, length = 20)
    private String cedula;


    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 120, message = "El nombre no debe superar los
120 caracteres")
    @Column(nullable = false, length = 120)
    private String nombre;


    @NotBlank(message = "El PIN es obligatorio")
    @Column(name = "pin_hash", nullable = false)
    private String pinHash;


    @Column(nullable = false, length = 20)
    private String rol = "empleado";
```

```
    @Column(nullable = false)
    private Boolean activo = true;
}
```

Java

None

```
//
src/main/java/com/nomina/portalnomina/controller/UsuarioContro
ller.java (Extracto)
@RestController
@RequestMapping("/api/usuarios")
public class UsuarioController {

    @Autowired
    private UsuarioService usuarioService;

    @PostMapping("/crear")
    public ResponseEntity<?> crearUsuario(@RequestBody Usuario
usuario) {
        try {
```

```
            Usuario usuarioGuardado =
usuarioService.registrarUsuario(usuario);

            return new ResponseEntity<>(usuarioGuardado,
HttpStatus.CREATED);

        } catch (DataIntegrityViolationException e) {

            Map<String, String> error = new HashMap<>();

            error.put("error", "Conflicto: La cédula ingresada
ya se encuentra registrada.");

            return new ResponseEntity<>(error,
HttpStatus.CONFLICT);

        }

    }



    @DeleteMapping("/eliminar/{cedula}")

    public ResponseEntity<?> eliminarUsuario(@PathVariable
String cedula) {

        try {

            usuarioService.eliminarUsuarioLogico(cedula);

            Map<String, String> respuesta = new HashMap<>();

            respuesta.put("mensaje", "Usuario inhabilitado
(borrado lógico) correctamente.");
```

```
            return new ResponseEntity<>(respuesta,
HttpStatus.OK);

        } catch (RuntimeException e) {

            Map<String, String> error = new HashMap<>();

            error.put("error", e.getMessage());

            return new ResponseEntity<>(error,
HttpStatus.NOT_FOUND);

        }

    }



    @PostMapping("/login")

    public ResponseEntity<?> login(@RequestBody Map<String,
String> credenciales) {

        try {

            Usuario usuario =
usuarioService.autenticarUsuario(

                credenciales.get("cedula"),
credenciales.get("pin")

            );

            Map<String, Object> respuesta = new HashMap<>();

            respuesta.put("mensaje", "Autenticación exitosa");
```

```
            respuesta.put("cedula", usuario.getCedula());

            respuesta.put("nombre", usuario.getNombre());

            respuesta.put("rol", usuario.getRol());

            return new ResponseEntity<>(respuesta,
HttpStatus.OK);

        } catch (IllegalArgumentException e) {

            return new ResponseEntity<>(Map.of("error",
e.getMessage()), HttpStatus.BAD_REQUEST);

        } catch (SecurityException e) {

            return new ResponseEntity<>(Map.of("error",
e.getMessage()), HttpStatus.UNAUTHORIZED);

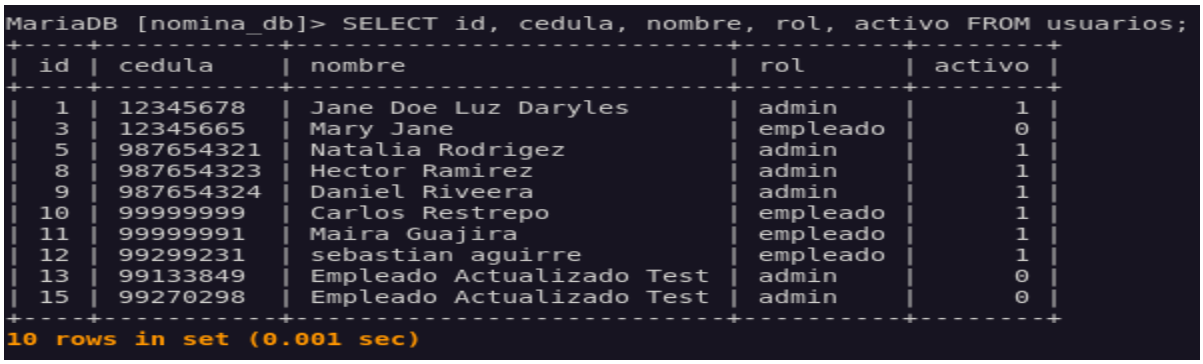
        }

    }

}
```

Apéndice D. Evidencias de ejecución (Postman, pruebas y base de datos)

Figura D1. *Verificación de tablas físicas y borrado lógico (*`activo = 0`*) en MariaDB*



```
MariaDB [nomina_db]> SELECT id, cedula, nombre, rol, activo FROM usuarios;
+----+-----------+----------------------------+----------+--------+
| id | cedula    | nombre                     | rol      | activo |
+----+-----------+----------------------------+----------+--------+
|  1 | 12345678  | Jane Doe Luz Daryles       | admin    |      1 |
|  3 | 12345665  | Mary Jane                  | empleado |      0 |
|  5 | 987654321 | Natalia Rodrigez           | admin    |      1 |
|  8 | 987654323 | Hector Ramirez             | admin    |      1 |
|  9 | 987654324 | Daniel Riveera             | admin    |      1 |
| 10 | 99999999  | Carlos Restrepo            | empleado |      1 |
| 11 | 99999991  | Maira Guajira              | empleado |      1 |
| 12 | 99299231  | sebastian aguirre          | empleado |      1 |
| 13 | 99133849  | Empleado Actualizado Test  | admin    |      0 |
| 15 | 99270298  | Empleado Actualizado Test  | admin    |      0 |
+----+-----------+----------------------------+----------+--------+
10 rows in set (0.001 sec)
```

Nota. Fuente: Elaboración propia. Evidencia de persistencia relacional, cifrado `bcrypt` en `pin_hash` e inhabilitación mediante borrado lógico (`activo = 0`).

Apéndice E. Historial de commits del repositorio

Figura E1. *Historial de commits atómicos bajo el estándar Conventional Commits en Git*

```
71b6675 (HEAD -> main, origin/main, origin/adrian_dev, adrian_dev) feat: corte vertical añadido y probado
9f3c6e1 feat: crud completo, puesta en orden excepciones y errores 400
91012d6 feat: implementar CRUD: actualizar y borrado logico.
d86bdc6 feat: implementacion de CRUD: crear y leer. Para Usuario
9842783 docs: documentos complementarios parte 1
5c01e6b fixgit commit -m 'feat: update compiled classes for PortalNominaApplication, HealthController, Usuario, and UsuarioRepository'
586d185 feat(backend): completa esqueleto ejecutable de Spring Boot con conexion verificada a MariaDB
67d6c8e feat: estrucutra springboot puesta
114966e docs(adr): agrega ADR-002 (autenticación JWT) y ADR-003 (portal web responsivo)
25a75f3 docs: ADR-001 agregado
9c4d02f docs: nuevas adicciones tarea
fd48824 docs: correciones menores
31c4164 agregar punto 3 al proyecto
95b90e9 feat: esquema inicial agregado punto 5.2
:...skipping...
```

Nota. Fuente: Elaboración propia. Registro cronológico del repositorio oficial https://github.com/AdrianALM766/Entregable_Nomina.git.